CSD302: Design and Analysis of Algorithms

Today’s Lecture…

- Euclid’s Algorithm for GCD
- Extended Euclid’s Algorithm
- Application

Greatest Common Divisor (GCD)

- The greatest common divisor of two integers a and b that are not both 0 is a common divisor d > 0 of a and b such that all other common divisors of a and b divide d.

- Notation:

$$d = \gcd(a, b) \text{ or } d = (a,b)$$

Greatest Common Divisor

- Simple Factoring Method

Greatest Common Divisor

- Simple Factoring Method
  - Input: Two integers ***a*** and **b**
  - Output: Largest integer that divides both, d

Greatest Common Divisor

- Simple Factoring Method
  - Input: Two integers ***a*** and **b**
  - Output: Largest integer that divides both, d

Algorithm:

1. Prime factorization of a: a_1, a_2, … a_k s.t.

$$a = a_1 . a_2 \ldots . a_k$$

Greatest Common Divisor

- Simple Factoring Method
  - Input: Two integers ***a*** and **b**
  - Output: Largest integer that divides both, d

Algorithm:

1. Prime factorization of a: $a_1, a_2, \ldots a_k$ s.t.

$$a = a_1 . a_2 \ldots . a_k$$

2. Prime factorization of b: $b_1, b_2, \ldots b_l$ s.t.

$$b = b_1 . b_2 . \ldots . b_l$$

Greatest Common Divisor

- Simple Factoring Method
  - Input: Two integers ***a*** and **b**
  - Output: Largest integer that divides both, d

Algorithm:

1. Prime factorization of a: $a_1, a_2, \ldots a_k$ s.t.

$$a = a_1 . a_2 \ldots . a_k$$

2. Prime factorization of b: $b_1, b_2, \ldots b_l$ s.t.

$$b = b_1 . b_2 . \ldots . b_l$$

3. Identify common factors, multiply and return

Greatest Common Divisor

- Example: Simple Factoring Method
  - Input: Two integers **154** and **462**
  - Output: Largest integer that divides both

Steps:

1. 154 = 2.7.11
2. 682 = 2.11.31
3. Common factors: 2, 11

   GCD = 2.11 = 22

Greatest Common Divisor

- Example: Simple Factoring Method
  - Input: Two integers **154** and **462**
  - Output: Largest integer that divides both

Steps:

1. 154 = 2.7.11 …. Divisions 2, 3, 5, … 11
2. 682 = 2.11.31
3. Common factors: 2, 11

   GCD = 2.11 = 22

Greatest Common Divisor

- Example: Simple Factoring Method
  - Input: Two integers **154** and **462**
  - Output: Largest integer that divides both

Steps:

1. 154 = 2.7.11 …. Divisions 2, 3, 5, … 11
2. 682 = 2.11.31 …. Divisions 2, 3, 5, … 31
3. Common factors: 2, 11

   GCD = 2.11 = 22

Greatest Common Divisor

- Observation I: A common divisor d of the integers a and b divides the difference a − b.

Greatest Common Divisor

- Observation I: A common divisor d of the integers a and b divides the difference a − b.

- Why?

Greatest Common Divisor

- Observation I: A common divisor d of the integers a and b divides the difference $a - b$.

Why?

- Suppose $a = q_1 d$ and $b = q_2 d$ where $q_1, q_2 \in \mathbb{Z}$

Greatest Common Divisor

- Observation I: A common divisor d of the integers a and b divides the difference $a - b$.

Why?

- Suppose $a = q_1 d$ and $b = q_2 d$ where $q_1, q_2 \in \mathbb{Z}$

then $a - b = (q_1 - q_2)d$; hence, $d \mid (a - b)$.

Euclid's Algorithm

- Input: a, b positive integers
- Output: d = gcd(a, b)

1. while($a \neq$ b) do
2. (a, b) = (min(a, b), max(a, b) − min(a, b));
3. return a;

Example: a = 34 and b = 19

Then the value of the variables a and b in each iteration is

1. (a, b) = (34, 19)
2. (a, b) = (19, 15)
3. (a, b) = (15, 4)
4. (a, b) = (4, 11)
5. (a, b) = (4, 7)
6. (a, b) = (4, 3)
7. (a, b) = (3, 1)
8. (a, b) = (1, 2)
9. (a, b) = (1, 1)

Therefore, gcd(34, 19) = gcd(1, 1) = 1.

Euclid's Algorithm

- Input: a, b positive integers
- Output: d = gcd(a, b)

```
while (b ≠ 0) do
        (a, b) = (min (a, b), max (a, b) % min (a, b));
return a;
```

Example: a = 34 and b = 19

Then the value of the variables a and b in each iteration is

1. (a, b) = (34, 19)
2. (a, b) = (19, 15)
3. (a, b) = (15, 4)
4. (a, b) = (4, 3)
5. (a, b) = (3, 1)
6. (a, b) = (1, 0)

Therefore, gcd(34, 19) = gcd(1, 1) = 1.

Theorem

- Given integers a and b with $b \neq 0$, there exists unique integers q and r such that $a = qb + r$ and $r < b$.

Theorem

- Given integers a and b with $b \neq 0$, there exists unique integers q and r such that $a=qb+r$ and $r<b$. The number q is called the quotient, while r is called the remainder. Then the equality

$$\gcd(a,b)=\gcd(b,r) \text{ holds.}$$

Theorem

- Given integers a and b with $b \neq 0$, there exists unique integers q and r such that $a=qb+r$ and $r<b$. The number q is called the quotient, while r is called the remainder. Then the equality

  $$\gcd(a,b)=\gcd(b,r) \text{ holds.}$$

- Proof: Let $d=\gcd(a,b)$, $d1=\gcd(b,r)$ where $r = a \bmod b$

Theorem

- Given integers a and b with $b \neq 0$, there exists unique integers q and r such that $a=qb+r$ and $r<b$. The number q is called the quotient, while r is called the remainder. Then the equality

  $$\gcd(a,b)=\gcd(b,r) \text{ holds.}$$

- Proof: Let $d=\gcd(a,b)$, $d1=\gcd(b,r)$.

  Since $a = qb + r$; $r= a - qb$, it follows that $d|r$, then $d| d1$.

Theorem

- Given integers a and b with $b \neq 0$, there exists unique integers q and r such that $a=qb+r$ and $r<b$. The number q is called the quotient, while r is called the remainder. Then the equality

  $$\gcd(a,b)=\gcd(b,r) \text{ holds.}$$

- Proof: Let $d=\gcd(a,b)$, $d1=\gcd(b,r)$.

  Since $a = qb + r$; $r= a - qb$, it follows that $d|r$, then $d| d1$.

  Similarly, $d1| a$, and $d1 | b$ hold, thus $d1 | d$. Now $d1 | d$ and $d | d1$.

  Thus, $\gcd(a,b)=\gcd(b,r)$.

Euclid Algorithm - GCD(a, b)

Euclid's Theorem: Let a, b > 0 be positive integers. If b divides a then GCD(a,b)= b else GCD(a,b)=(b, a % b)

Algorithm - GCD(a, b)

Input: Positive integers a, b such that $b \leq a$;
Output GCD of a, b ;
Let $c = a \mod b$;
if $c = 0$ **then**
  return b
else
  return Euclid-GCD(b, c)

Euclid’s algorithm: Analysis

Bit Computational Model

- Counting the number of bit operations needed for the computation
- First computing the number of iterations in the worst case

Observation:

- The number a % b ≤ $a/2$, i.e. the size of a mod b is strictly less than $|a|$.

Observation:

- The number a (mod b) $\leq a/2$, i.e. the size of a mod b is strictly less than $|a|$.

Proof:

Case-I: b $\leq$ $a/2$, in which case a (mod b) $\leq$ b $\leq$ $a/2$.

Observation:

- The number a (mod b) $\leq a/2$, i.e. the size of a mod b is strictly less than $|a|$.

Proof:

Case-I: b $\leq$ $a/2$, in which case a (mod b) $\leq$ b $\leq a/2$.

Case-II: b > $a/2$, in which case a (mod b) = a - b $\leq a/2$.

Observation:

- The number a (mod b) $\leq a/2$, i.e. the size of a mod b is strictly less than $|a|$.

Proof:

Case-I: b $\leq$ $a/2$, in which case a (mod b) $\leq$ b $\leq a/2$.

Case-II: b > $a/2$, in which case a (mod b) = a - b $\leq a/2$.

In two recursive calls the first argument will be a mod b, which is smaller than $a/2$

Observation:

- The number a (mod b) $\leq a/2$, i.e. the size of a mod b is strictly less than $|a|$.

Proof:

Case-I: b $\leq$ $a/2$, in which case a (mod b) $\leq$ b $\leq a/2$.

Case-II: b > $a/2$, in which case a (mod b) = a - b $\leq a/2$.

In two recursive calls the first argument will be a mod b, which is smaller than $a/2$.

- It follows that the Euclid's algorithm is bounded by $|a|$, or equivalently 2(log a) recursive calls.
- Taking n = $|a|$ + |b|, number of recursive calls in the computation is $O(n)$.

Euclid Algorithm: Worst-case Analysis

- # iterations O(n)
- As *a* and b, represented by n-bits,
- The long division method to compute mod (which is really a division) can be done with $O(n^2)$ bit operations.
- $O(n^2)$ operations in each recursive call.
- We obtain the running time: $O(n) \times O(n^2) = O(n^3)$.

Euclid's Algorithm

- Computes d= gcd(a,b)
- Also provides two integers x and y such that

$$xa + yb = d$$

Euclid's Algorithm

- Computes d= gcd(a,b)
- Also provides two integers x and y such that

$$xa + yb = d$$

Note: To verify d as the GCD (a, b), necessary to check:

1. d | a, d | b and
2. $xa + yb = d$

Theorem

$$\gcd(a, b) = \min\{xa + yb \mid xa + yb > 0\}$$

- Proof: Let $l = \min\{xa + yb \mid xa + yb > 0\}$.

  Clearly $\gcd(a, b)$ divides l and hence $\gcd(a, b) \leq l$.

Theorem

$$\gcd(a, b) = \min\{xa + yb \mid xa + yb > 0\}$$

- Proof: Let $l = \min\{xa + yb \mid xa + yb > 0\}$.

  Clearly $\gcd(a, b)$ divides l and hence $\gcd(a, b) \leq l$.

  - We now prove that l divides a (also b).

    Let us prove by contradiction that

    $a = lq + r$ where $l > r > 0$.

Theorem

$$\gcd(a, b) = \min\{xa + yb \mid xa + yb > 0\}$$

- Proof: Let $l = \min\{xa + yb \mid xa + yb > 0\}$.

  Clearly $\gcd(a, b)$ divides l and hence $\gcd(a, b) \leq l$.

  - We now prove that l divides a (also b).

    Let us prove by contradiction that

    $$a = lq + r \text{ where } l > r > 0.$$

    Now $r = a - lq = a - (xa+yb)q$

    $$= (1 - xq)a - (yq)b$$

Theorem

$$\gcd(a, b) = \min\{xa + yb \mid xa + yb > 0\}$$

- Proof: Let $l = \min\{xa + yb \mid xa + yb > 0\}$.

  Clearly $\gcd(a, b)$ divides l and hence $\gcd(a, b) \leq l$.

  - We now prove that l divides a (also b).

    Let us prove by contradiction that

    $$a = lq + r \text{ where } l > r > 0.$$

    Now $r = a - lq = a - (xa+yb)q$

    $$= (1 - xq)a - (yq)b$$

  It shows that r is in the set $\{xa + yb\}$ and $r > 0$

  Contradicting the minimality of l.

Extended Euclid's Algorithm

- Application: computing the inverse in a multiplicative prime field F_q^* where q is prime.

  F_q^* = {1, 2 . . . (q − 1)}

  where the multiplication is performed modulo q.

Extended Euclid's Algorithm

- Input: a, b positive integers
- Output: d = gcd(a, b), the numbers x and y

Extended Euclid's Algorithm

- Input: a, b positive integers
- Output: x, y of $xa+yb=\gcd(a,b)$.
  - If $b=0$, then $\gcd(a,b)=a$, $x=1, y=0$.

Extended Euclid's Algorithm

- Input: a, b positive integers
- Output: x, y of $xa+yb=\gcd(a,b)$.
  - If $b=0$, then $\gcd(a,b)=a$, $x=1, y=0$.
  - If $b\neq 0$, then
    - $a= q_0 b+r_0$
    - $b=q_1 r_0+r_1$
    - $r_0=q_2 r_1+r_2$

      ……
    - $r_{k-3}=q_{k-1} r_{k-2}+r_{k-1}$
    - $r_{k-2}=q_k r_{k-1}+r_k$
    - $r_{k-1}=q_{k+1} r_k+0$ where $r_k = \gcd(a,b)$

Extended Euclid's Algorithm

- Input: a, b positive integers
- Output: x, y of $xa+yb=\gcd(a,b)$.
  - If $b=0$, then $\gcd(a,b)=a$, $x=1, y=0$.
  - If $b\neq 0$, then
    - $a= q_0 b+r_0$
    - $b=q_1 r_0+r_1$
    - $r_0=q_2 r_1+r_2$

      ……
    - $r_{k-3}=q_{k-1} r_{k-2}+r_{k-1}$
    - $r_{k-2}=q_k r_{k-1}+r_k$
    - $r_{k-1}=q_{k+1} r_k+0$ where $r_k = \gcd(a,b)$

$\Rightarrow$ $r_k = r_{k-2} - q_k r_{k-1} = r_{k-2} - q_k (r_{k-3} - q_{k-1} r_{k-2})$ and so on…

Example: Extended Euclid's Algorithm

- $a = 84$ and $b = 33$.
- $\gcd(84, 33) = \gcd(33, 18) = \gcd(18, 15) = \gcd(15, 3) = \gcd(3, 0) = 3$

- We work backwards to write 3 as a linear combination of 84 and 33:
  - $3 = 18 - 15$ [Now 3 is a linear combination of 18 and 15]

Example: Extended Euclid's Algorithm

- $a = 84$ and $b = 33$.
- $\gcd(84, 33) = \gcd(33, 18) = \gcd(18, 15) = \gcd(15, 3) = \gcd(3, 0) = 3$

- We work backwards to write 3 as a linear combination of 84 and 33:
  - $3 = 18 - 15$ [Now 3 is a linear combination of 18 and 15]
    $= 18 - (33 - 18) = 2(18) - 33$ [Now 3 is a linear combination of 18 and 33]
    $= 2(84 - 2 \times 33)) - 33 = 2 \times 84 - 5 \times 33$ [Now 3 is a linear combination of 84 and 33]

Thus, $x = 2$ and $y = -5$

Definition

- If the remainder of *a* divided by q is equal to the remainder of b divided by q, then integers *a* and b are called congruent modulo q, denoted by

$$a \equiv b \pmod{q}$$

Definition

- Given a and q, if there exists an integer b that satisfies the equation

  $$ab \equiv 1 \pmod{q},$$

  then b is called the multiplicative inverse of a modulo q.

Note: $ab \equiv 1 \pmod{q}$ implies $q \mid (ab-1)$

Definition

- Given a and q, if there exists an integer b that satisfies the equation

  $ab \equiv 1 \pmod{q}$,

  then b is called the multiplicative inverse of a modulo q.

Note: $ab \equiv 1 \pmod{q}$ implies $q \mid (ab-1)$

Important Result: Solving the congruence $ax \equiv b \pmod{m}$ is equivalent to solving $ax - my = b$.

Definition

- Given a and q, if there exists an integer b that satisfies the equation

  $ab \equiv 1 \pmod{q}$,

  then b is called the multiplicative inverse of a modulo q.

Note: $ab \equiv 1 \pmod{q}$ implies $q \mid (ab-1)$

Important Result: Solving the congruence $ax \equiv b \pmod{m}$ is equivalent to solving $ax - my = b$.

- If $ax \equiv 1 \pmod{m}$ is equivalent to solve $ax - my = 1$

Theorem

- Given integers a and q, the necessary and sufficient condition for the existence of a multiplicative inverse of a modulo q is gcd(a,q)=1.

Proof:

if there exists an integer b with ab≡1(mod q),

then ab=kq+1 holds.

Since gcd(a,q)=d, d|1, then d=1.

Theorem

- If gcd(a,b)=d, then there exist integers x and y with ax +by=d. If d=1, that is ax+by=1, then x is the multiplicative inverse of a modulo b, y is the multiplicative inverse of b modulo a.

- Proof: As gcd (a,b)=d, thus a=md, b=nd, and gcd(m,n)=1 for some integer m and n.

  From previous result, there exist integers x and y with

  xm+yn=1.

Therefore, ax+by=mdx+ndy=(xm+yn)d=d, that is xa+yb=d.

Application to Cryptography

RSA Algorithm:

- Pick two large prime numbers p, q, and let $n = p \cdot q$.

Application to Cryptography

RSA Algorithm:

- Pick two large prime numbers p, q, and let $n = p \cdot q$.
- Take message as an integer m such that $0 \leq m < n$.

Application to Cryptography

RSA Algorithm:

- Pick two large prime numbers p, q, and let $n = p \cdot q$.
- Take message as an integer m such that $0 \leq m < n$.
- Let $\phi(n) = (p - 1) \cdot (q - 1)$, Eulier's totient function, and is the private key.

Application to Cryptography

RSA Algorithm:

- Pick two large prime numbers p, q, and let $n = p \cdot q$.
- Take message as an integer m such that $0 \leq m < n$.
- Let $\phi(n) = (p - 1) \cdot (q - 1)$, Eulier's totient function, and is the private key.
- Let d, e be integers such that

$$e \cdot d \equiv 1 \bmod \phi(n) \text{ where } e, d \text{ are coprimes to } \phi(n).$$

Application to Cryptography

RSA Algorithm:

- Pick two large prime numbers p, q, and let $n = p \cdot q$.
- Take message as an integer m such that $0 \leq m < n$.
- Let $\phi(n) = (p - 1) \cdot (q - 1)$, Eulier's totient function, and is the private key.
- Let d, e be integers such that

  $e \cdot d \equiv 1 \bmod \phi(n)$ where e, d are coprimes to $\phi(n)$.

- Encryption of m: Compute $m^e \pmod{n}$ and

Application to Cryptography

RSA Algorithm:

- Pick two large prime numbers p, q, and let $n = p \cdot q$.
- Take message as an integer m such that $0 \leq m < n$.
- Let $\phi(n) = (p - 1) \cdot (q - 1)$, Eulier's totient function, and is the private key.
- Let d, e be integers such that

  $e \cdot d \equiv 1 \bmod \phi(n)$ where e, d are coprimes to $\phi(n)$.

- Encryption of m: Compute $m^e \pmod{n}$ and
- Decryption: Compute $(m^e)^d \pmod{n}$

Application to Cryptography

RSA Algorithm:

- Encryption of m: Compute m^e mod n and
- Decryption: Compute $(m^e)^d$ mod n.

Application to Cryptography

RSA Algorithm:

- Encryption of m: Compute m^e mod n and
- Decryption: Compute $(m^e)^d$mod n.

Note:

- $(m^e)^d \equiv m^{k\varphi(n)+1} \bmod p \equiv m \bmod p$. [Euler theorem $a^{\phi(p)} \equiv 1 (\bmod p)$]

Application to Cryptography

RSA Algorithm:

- Encryption of m: Compute m^e mod n and
- Decryption: Compute $(m^e)^d$ mod n.

Note:

- $(m^e)^d \equiv m^{k\varphi(n)+1} \bmod p \equiv m \bmod p$. [Euler theorem $a^{\Phi(p)} \equiv 1 (\bmod p)$]
- Similarly $(m^e)^d \equiv m \bmod q$

$\Rightarrow$ $(m^e)^d \equiv m \bmod (pq) \equiv m \bmod n$ [from Chinese Remainder Theorem]

Underlying assumption: given n, e, it is hard to compute d.

Application to Cryptography

RSA Algorithm:

- The two main algorithmic steps are
  i. Computing exponentiation, which can be done using the divide and conquer technique
  ii. Computing the multiplicative inverse [$e \cdot d \equiv 1 \bmod \phi(n)$] which can be done using the Extended Euclid's algorithm.

Application to Cryptography

RSA Algorithm:

- The two main algorithmic steps are
  i. Computing exponentiation, which can be done using the divide and conquer technique
  ii. Computing the multiplicative inverse [$e \cdot d \equiv 1 \bmod \phi(n)$] which can be done using the Extended Euclid's algorithm.

$ed \equiv 1 \bmod \phi(n)$ is equivalent to solve: $ed + \phi(n)y = 1$

where $\gcd(e, \phi(n))=1$ and we know how to get d(private key) following Extended Euclid's algorithm.

References

- Design and Analysis of Algorithms :A Contemporary Perspective, Sandeep Sen and Amit Kumar
- An Introduction to Theory of Numbers, Ivan Niven, Herbert S. Zuckerman, Hugh L. Montgomery
- https://ieeexplore.ieee.org/abstract/document/5691644

Chinese Remainder Theorem

- Theorem: Let $m_1,\ldots, m_n$ be pairwise coprime (that is $\gcd(m_i,m_j) =1$, $i\neq j$). Then the system of n equations

$$x= a_1 \pmod{m_1}$$
$$\ldots\ldots$$
$$x= a_n \pmod{m_n}$$

has a unique solution for x modulo M where $M= m_1 x \ldots x\, m_n$.

$$x = \sum_{i=1}^{n} a_i b_i b'_i \pmod{M}$$

where $b_i=M/m_i$ (the product of all the moduli except for m_i)

and $b'_i=b_i^{-1} \pmod{m_i}$